

NNGraph DSL

A Domain-Specific Language for Neural Network Graph to PyTorch Code Conversion

Implemented with ANTLR4 • Inspired by the NADER Paper

1. Introduction

NNGraph DSL is a domain-specific language designed to express neural network computational graphs in a human-readable, declarative syntax. The compiler — built with ANTLR4 — parses an NNGraph source file and emits executable PyTorch Python code.

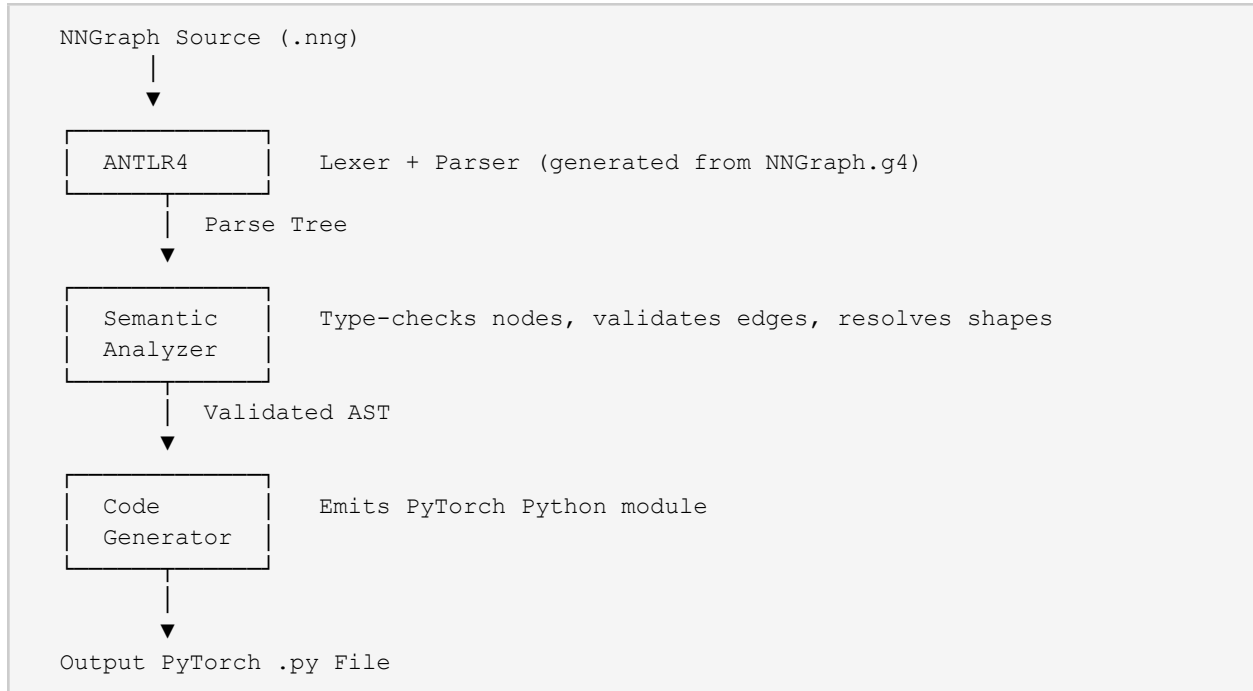
The design is motivated by the NADER framework (Neural Architecture Design via Representation), which represents neural network architectures as directed computational graphs. Once a valid graph is defined in the DSL, the toolchain automatically translates it into runnable PyTorch modules, removing the need to hand-code repetitive boilerplate.

1.1 Goals

- Express any feed-forward or branching neural network topology as a graph.
- Generate clean, idiomatic PyTorch code (
- Support common layer types, activations, normalization, and skip connections.
- Be implementation-agnostic — the DSL is the single source of truth.
- Allow inline hyperparameter configuration per node.

2. Architecture Overview

The toolchain follows a classic compiler pipeline:



3. Language Specification

3.1 Program Structure

An NNGraph program is composed of three top-level sections: a **model** declaration, a **graph** block defining nodes and edges, and an optional **config** block for training/export metadata.

```
model <ModelName> {
    input  <node_id> : tensor(<shape>)
    output <node_id>
}

graph {
    node <id> : <LayerType>(<params>)
    ...
    edge <src> -> <dst>
    ...
}

config {
    batch_size = <int>
}
```

```
device      = "cpu" | "cuda"
}
```

3.2 Data Types

DSL Type	Example	Description
int	128	Integer literal
float	0.1	Floating-point literal
bool	true / false	Boolean flag
string	"relu"	Quoted string
shape	(3, 224, 224)	Tuple of ints for tensor shape
None	None	Optional absent value

3.3 Supported Layer Types

DSL Keyword	PyTorch Mapping	Key Parameters
Linear	nn.Linear	in_features, out_features, bias
Conv2d	nn.Conv2d	in_ch, out_ch, kernel, stride, padding
Conv1d	nn.Conv1d	in_ch, out_ch, kernel, stride
BatchNorm2d	nn.BatchNorm2d	num_features
LayerNorm	nn.LayerNorm	normalized_shape
MaxPool2d	nn.MaxPool2d	kernel, stride
AvgPool2d	nn.AvgPool2d	kernel, stride
Dropout	nn.Dropout	p (probability)
Flatten	nn.Flatten	start_dim, end_dim
Embedding	nn.Embedding	num_embeddings, embedding_dim
MultiHeadAttn	nn.MultiheadAttention	embed_dim, num_heads
LSTM	nn.LSTM	input_size, hidden_size, num_layers
GRU	nn.GRU	input_size, hidden_size

3.4 Activation Functions

Activations are expressed as special single-parameter nodes. They map directly to `nn.Module` wrappers in PyTorch:

```

node act1 : ReLU()
node act2 : Sigmoid()
node act3 : Tanh()
node act4 : GELU()
node act5 : Softmax(dim=1)
node act6 : LeakyReLU(negative_slope=0.01)
node act7 : ELU(alpha=1.0)

```

3.5 Special Operations

Non-parameterized graph operations are supported as built-in node types:

```

node add1 : Add()           // element-wise sum of all incoming edges
node cat1 : Concat(dim=1)   // concatenation along a dimension
node res1 : Residual()      // shortcut add (exactly 2 incoming edges required)
node split : Split(chunks=2, dim=1) // split tensor into N chunks

```

3.6 Edge Syntax

Edges define the data-flow between nodes. A simple directed edge is written with `->`. Edges may be labeled for documentation purposes:

```

edge conv1 -> bn1           // simple edge
edge bn1    -> relu1
edge relu1  -> fc1 [label="main_path"] // labeled edge (optional)

// Multiple inputs to Add or Concat:
edge conv1 -> add1
edge conv2 -> add1

```

3.7 Comments

```

// This is a single-line comment

/* This is a
   multi-line comment */

```

4. Worked Examples

Example 1 — Simple MLP Classifier

A three-layer multi-layer perceptron for 10-class classification from 784-dimensional input (e.g., MNIST).

NNGraph Source

```
model MLP {
    input  x : tensor(784)
    output out
}

graph {
    node fc1      : Linear(in_features=784, out_features=256)
    node relu1    : ReLU()
    node drop1    : Dropout(p=0.3)
    node fc2      : Linear(in_features=256, out_features=128)
    node relu2    : ReLU()
    node fc3      : Linear(in_features=128, out_features=10)
    node out      : Softmax(dim=1)

    edge x      -> fc1
    edge fc1    -> relu1
    edge relu1  -> drop1
    edge drop1  -> fc2
    edge fc2    -> relu2
    edge relu2  -> fc3
    edge fc3    -> out
}

config {
    batch_size = 64
    device      = "cuda"
}
```

Generated PyTorch Code

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256)
        self.relu1 = nn.ReLU()
        self.drop1 = nn.Dropout(p=0.3)
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(in_features=128, out_features=10)
        self.out = nn.Softmax(dim=1)
```

```

def forward(self, x):
    x = self.fc1(x)
    x = self.relu1(x)
    x = self.drop1(x)
    x = self.fc2(x)
    x = self.relu2(x)
    x = self.fc3(x)
    x = self.out(x)
    return x

if __name__ == '__main__':
    device = torch.device('cuda')
    model = MLP().to(device)
    x = torch.randn(64, 784).to(device)
    print(model(x).shape)  # → torch.Size([64, 10])

```

Example 2 — ResNet-Style Block with Skip Connection

A convolutional residual block: the input is convolved twice (Conv → BN → ReLU → Conv → BN), then added back to the original input via a skip connection.

NNGraph Source

```

model ResBlock {
    input  x : tensor(64, 32, 32)    // C=64, H=32, W=32
    output out
}

graph {
    node conv1  : Conv2d(in_ch=64, out_ch=64, kernel=3, stride=1, padding=1)
    node bn1    : BatchNorm2d(num_features=64)
    node relu1  : ReLU()
    node conv2  : Conv2d(in_ch=64, out_ch=64, kernel=3, stride=1, padding=1)
    node bn2    : BatchNorm2d(num_features=64)
    node skip   : Residual()          // adds skip + main path
    node relu2  : ReLU()
    node out    : Flatten()

    // Main path
    edge x      -> conv1
    edge conv1  -> bn1
    edge bn1    -> relu1
    edge relu1  -> conv2
    edge conv2  -> bn2

    // Skip + merge
    edge bn2    -> skip
    edge x      -> skip    [label="shortcut"]
}

```

```

    edge skip -> relu2
    edge relu2 -> out
}

```

Generated PyTorch Code

```

import torch
import torch.nn as nn

class ResBlock(nn.Module):
    def __init__(self):
        super(ResBlock, self).__init__()
        self.conv1 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu1 = nn.ReLU()
        self.conv2 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.relu2 = nn.ReLU()
        self.out = nn.Flatten()

    def forward(self, x):
        identity = x          # shortcut branch
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu1(x)
        x = self.conv2(x)
        x = self.bn2(x)
        x = x + identity      # Residual merge
        x = self.relu2(x)
        x = self.out(x)
        return x

```

Example 3 — Multi-Branch Inception-Style Block

A block with two parallel convolutional branches of different kernel sizes, whose outputs are concatenated along the channel dimension.

NNGraph Source

```

model InceptionBlock {
    input  x : tensor(32, 56, 56)
    output out
}

graph {
    // Branch A - 1x1 conv
    node convA1 : Conv2d(in_ch=32, out_ch=16, kernel=1, stride=1, padding=0)
    node reluA  : ReLU()

```

```

// Branch B - 3x3 conv
node convB1 : Conv2d(in_ch=32, out_ch=16, kernel=1, stride=1, padding=0)
node convB2 : Conv2d(in_ch=16, out_ch=32, kernel=3, stride=1, padding=1)
node reluB  : ReLU()

// Merge branches
node cat1    : Concat(dim=1)    // 16+32 = 48 channels
node bn1     : BatchNorm2d(num_features=48)
node out     : ReLU()

// Branch A wiring
edge x       -> convA1
edge convA1  -> reluA

// Branch B wiring
edge x       -> convB1
edge convB1  -> convB2
edge convB2  -> reluB

// Merge
edge reluA   -> cat1
edge reluB   -> cat1
edge cat1    -> bn1
edge bn1     -> out
}

```

Generated PyTorch Code

```

import torch
import torch.nn as nn

class InceptionBlock(nn.Module):
    def __init__(self):
        super(InceptionBlock, self).__init__()
        # Branch A
        self.convA1 = nn.Conv2d(32, 16, kernel_size=1, stride=1, padding=0)
        self.reluA = nn.ReLU()
        # Branch B
        self.convB1 = nn.Conv2d(32, 16, kernel_size=1, stride=1, padding=0)
        self.convB2 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
        self.reluB = nn.ReLU()
        # Merge
        self.bn1 = nn.BatchNorm2d(48)
        self.out = nn.ReLU()

    def forward(self, x):
        # Branch A
        a = self.convA1(x)
        a = self.reluA(a)

```



```

# Branch B
b = self.convB1(x)
b = self.convB2(b)
b = self.reluB(b)
# Concat and finalize
x = torch.cat([a, b], dim=1) # shape: (batch, 48, 56, 56)
x = self.bn1(x)
x = self.out(x)
return x

```

Example 4 — Transformer Encoder Block

A single transformer encoder layer with multi-head self-attention, layer normalization, and a feed-forward sub-network.

NNGraph Source

```

model TransformerEncoder {
    input  x : tensor(512, 128)    // seq_len=512, d_model=128
    output out
}

graph {
    // Self-attention sub-layer
    node attn    : MultiHeadAttn(embed_dim=128, num_heads=8)
    node drop1   : Dropout(p=0.1)
    node norm1   : LayerNorm(normalized_shape=(128))

    // Feed-forward sub-layer
    node ff1     : Linear(in_features=128, out_features=512)
    node gelu1   : GELU()
    node drop2   : Dropout(p=0.1)
    node ff2     : Linear(in_features=512, out_features=128)
    node norm2   : LayerNorm(normalized_shape=(128))
    node out     : Flatten(start_dim=0, end_dim=1)

    // Attention path + residual
    edge x      -> attn
    edge attn   -> drop1
    edge drop1  -> norm1
    edge x      -> norm1    [label="residual_1"]

    // FFN path + residual
    edge norm1  -> ff1
    edge ff1    -> gelu1
    edge gelu1  -> drop2
    edge drop2  -> ff2
    edge ff2    -> norm2
    edge norm1  -> norm2    [label="residual_2"]
}

```

```
    edge norm2 -> out
}
```

Generated PyTorch Code

```
import torch
import torch.nn as nn

class TransformerEncoder(nn.Module):
    def __init__(self):
        super(TransformerEncoder, self).__init__()
        self.attn = nn.MultiheadAttention(embed_dim=128, num_heads=8,
batch_first=True)
        self.drop1 = nn.Dropout(p=0.1)
        self.norm1 = nn.LayerNorm(128)
        self.ff1 = nn.Linear(128, 512)
        self.gelu1 = nn.GELU()
        self.drop2 = nn.Dropout(p=0.1)
        self.ff2 = nn.Linear(512, 128)
        self.norm2 = nn.LayerNorm(128)
        self.out = nn.Flatten(start_dim=0, end_dim=1)

    def forward(self, x):
        # Self-attention sub-layer
        attn_out, _ = self.attn(x, x, x)
        attn_out = self.drop1(attn_out)
        x = self.norm1(x + attn_out) # residual_1

        # Feed-forward sub-layer
        ff_out = self.ff1(x)
        ff_out = self.gelu1(ff_out)
        ff_out = self.drop2(ff_out)
        ff_out = self.ff2(ff_out)
        x = self.norm2(x + ff_out) # residual_2
        x = self.out(x)
        return x
```

5. Semantic Analysis Rules

The semantic analyzer validates the parse tree before code generation. It enforces the following rules:

Rule	Description
No orphan nodes	Every declared node (except input/output) must appear in at least one edge.
No undefined references	Edge endpoints must reference declared node IDs.

Input reachability	Every node must be reachable from the input node following edge directions.
Output reachability	The output node must be reachable from at least one path from input.
Cycle detection	The graph must be a DAG. Cycles are rejected (RNN state is modeled separately).
Residual arity	A Residual() node must have exactly 2 incoming edges.
Parameter type check	Each layer parameter is validated against its expected type (int, float, bool, etc.).
Unique node IDs	Node identifiers must be unique within the graph block.

6. Code Generation Strategy

The code generator performs a topological sort of the validated DAG, then emits Python according to the following mapping strategy:

6.1 Topological Sort

Nodes are emitted in topological order — each node is guaranteed to have all its inputs computed before it executes. This makes the `forward()` method trivially linear in the simple case and uses branch variables for multi-path graphs.

6.2 Branch Variable Naming

When the DAG has branch points (a node with multiple outgoing edges), the code generator assigns branch-local variables. The branch is named after the source node:

```
a = self.branchA(x)      # first outgoing branch from x
b = self.branchB(x)      # second outgoing branch from x
x = torch.cat([a, b], dim=1)
```

6.3 Special Node Handling

Node Type	Generated Code Pattern
Add()	<code>x = a + b</code> (or sum of all inputs)
Concat(dim=d)	<code>x = torch.cat([a, b, ...], dim=d)</code>
Residual()	<code>x = main_path + shortcut</code> (identity saved before first conv)
Split(chunks=n, dim=d)	<code>parts = torch.chunk(x, n, dim=d)</code>

MultiHeadAttn	out, _ = self.attn(x, x, x) (self-attention)
LSTM / GRU	out, _ = self.lstm(x) (hidden state discarded by default)

7. Error Messages & Diagnostics

The compiler produces structured error messages with source line information. Examples:

```
// Undefined node reference in edge
Error [line 14]: Edge references undefined node 'fc99'.
  Hint: Declared nodes are: [x, fc1, relu1, fc2, out]

// Cycle detected
Error [line 22]: Cycle detected involving nodes: fc1 -> relu1 -> fc1.
  Hint: NNGraph graphs must be acyclic DAGs.

// Wrong arity for Residual
Error [line 18]: Node 'skip' (Residual) has 3 incoming edges; expected exactly 2.

// Type mismatch
Error [line 9]: Parameter 'p' of Dropout expects float, got string "0.3".

// Orphan node
Warning [line 6]: Node 'bn_unused' is declared but never referenced in any edge.
```

8. Toolchain Setup

8.1 Prerequisites

- Python 3.9+
- Java 11+ (required by ANTLR tool)
- ANTLR 4.13+ (antlr-4.13.x-complete.jar)
- antlr4-python3-runtime (pip install antlr4-python3-runtime)
- PyTorch 2.x (pip install torch)

8.2 Generate Lexer and Parser

```
# Download ANTLR jar (once)
curl -O https://www.antlr.org/download/antlr-4.13.1-complete.jar

# Generate Python3 lexer/parser from grammar
java -jar antlr-4.13.1-complete.jar \
```

```
-Dlanguage=Python3 \  
-visitor \  
-o generated/ \  
NNGraph.g4
```

8.3 Compile an NNGraph File

```
python nnggraph_compiler.py my_model.nng --output my_model.py
```

```
# Optionally validate generated code  
python -c "import my_model; print('OK')"
```

9. Future Extensions

- Shape inference — propagate tensor shapes through the graph and catch mismatches at compile time.
- Quantization annotations — add a
- ONNX export — emit
- Training scaffold — generate a full training loop with optimizer and loss from
- Visual graph rendering — output a Graphviz
- Macros / templates — allow reusable named sub-graphs (e.g., a named ResBlock that can be instantiated multiple times).
- Dynamic control flow — conditional branches expressed with an

Appendix A — Quick Reference Card

Construct	Syntax
Model declaration	model <Name> { input x : tensor(<s>) ; output <id> }
Graph block	graph { ... }
Node declaration	node <id> : <LayerType>(<params>)
Simple edge	edge <src> -> <dst>
Labeled edge	edge <src> -> <dst> [label="name"]
Config block	config { batch_size = 32 ; device = "cuda" }
Line comment	// comment
Block comment	/* comment */

— *End of Document* —